

ΠΡΟΓΡΑΜΜΑΤΙΣΤΙΚΗ ΑΣΚΗΣΗ

Η ΓΛΩΣΣΑ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ GREEK++

ΜΕΤΑΦΡΑΣΤΕΣ ΑΚΑΔΗΜΑΪΚΟ ΈΤΟΣ 2024-2025

ΑΘΗΝΑ ΣΤΑΣΙΝΟΥ ΑΜ:5355

ΑΡΕΤΗ ΚΙΡΠΙΤΣΑ ΑΜ:5255

ΠΕΡΙΕΧΟΜΕΝΑ

1. ΕΙΣΑΓΩΓΗ

1.1 Περιγραφή Greek++

1.2 Περιγραφή Assembly RISC-V

1.3 Μεταφραστής Greek++

2. ΛΕΚΤΙΚΗ ΑΝΑΛΥΣΗ

2.1 Λεκτική μονάδα

2.2 Βασικές βοηθητικές συναρτήσεις για τον Lexer

2.3 Κλάση Lex

3. ΣΥΝΤΑΚΤΙΚΗ ΑΝΑΛΥΣΗ

3.1 Κλάση Parser

3.2 Βοηθητικές συναρτήσεις για τον Parser

4. ΠΑΡΑΓΩΓΗ ΕΝΔΙΑΜΕΣΟΥ ΚΩΔΙΚΑ

4.1 QuadList βοηθητικές συναρτήσεις

4.2 Global βοηθητικές συναρτήσεις

4.3 Παραγωγή ενδιάμεσου κώδικα

5. ΠΙΝΑΚΑΣ ΣΥΜΒΟΛΩΝ ΚΑΙ ΣΗΜΑΣΙΟΛΟΓΙΚΗ ΑΝΑΛΥΣΗ

5.1 Περιγραφή της οργάνωσης του πίνακα συμβόλων

5.2 Υλοποίηση του πίνακα συμβόλων

5.3 Ενημέρωση της κλάσης Parser για τον πίνακα συμβόλων

5.4 Συμπέρασμα της λειτουργίας του πίνακα συμβόλων

5.5 Σημασιολογική Ανάλυση

6. ΠΑΡΑΓΩΓΗ ΤΕΛΙΚΟΥ ΚΩΔΙΚΑ

6.1 Βοηθητικές συναρτήσεις τελικού κώδικα

6.2 Υλοποίηση τελικού κώδικα

6.3 Παράδειγμα εκτέλεσης παραγόμενου αργείου στον RARS

1. ΕΙΣΑΓΩΓΗ

Ένας μεταγλωττιστής είναι ένα πρόγραμμα που μπορεί να διαβάσει ένα πρόγραμμα σε μία γλώσσα “πηγαία γλώσσα” και να το μεταφράσει σε ένα ισοδύναμο πρόγραμμα σε μία άλλη “γλώσσα στόχο”.



Σε αυτήν την εργασία στοχεύουμε στην μεταγλώττιση της **Greek++** (πηγαία γλώσσα) στην συμβολική γλώσσα μηχανής **Assembly** του επεξεργαστή RISC-V (γλώσσα στόχος). Συγκεκριμένα:

1.1 ΠΕΡΙΓΡΑΦΗ GREEK++

Η **Greek++** είναι μια γλώσσα προγραμματισμού βασισμένη στην ελληνική γλώσσα, τα αρχεία της είναι της μορφής « **filename.gr** ». Υποστηρίζει ακέραιους αριθμούς, συναρτήσεις, διαδικασίες, μετάδοση παραμέτρων με αναφορά και τιμή, δομές απόφασης και βρόχων καθώς και αναδρομικές κλήσεις. Πιο αναλυτικά:

- **Γράμματα** : μικρά και κεφαλαία γράμματα της ελληνικής και της αγγλικής αλφαβήτου
- **Αριθμοί** : 0 - 9
- **Πράξεις** : “+” “-” “*” “/”
- **Τελεστές συσχέτισης** : “<” “>” “=” “<=” “>=” “<>”
- **Ανάθεση** : “:=”
- **Διαχωριστές** : “;” “,” “:”
- **Σύμβολα ομαδοποίησης** : “(” “)” “[” “]”
- **Σχόλια** : “{” “}”
- **Πέρασμα παραμέτρων με αναφορά** : “%”
- **Δεσμευμένες λέξεις** : “πρόγραμμα”, “δήλωση”, “εάν”, “τότε”, “αλλιώς”, “εάν_τέλος”, “επανάλαβε”, “μέχρι”, “όσο”, “όσο_τέλος”, “για”, “έως”, “με_βήμα”, “για_τέλος”, “διάβασε”, “γράψε”, “συνάρτηση”, “διαδικασία”, “διαπροσωπεία”, “είσοδος”, “έξοδος”, “αρχή_συνάρτησης”, “τέλος_συνάρτησης”, “αρχή_διαδικασίας”, “τέλος_διαδικασίας”, “αρχή_προγράμματος”, “τέλος_προγράμματος”, “ή”, “και”, “εκτέλεσε”.

Περιορισμοί:

- **Συμβολοσειρές** : πρέπει να αποτελούνται από γράμματα ή ψηφία, αρχίζοντας υποχρεωτικά από γράμμα, το μέγεθός του δεν πρέπει να ξεπερνά τους τριάντα χαρακτήρες.
- **Αριθμοί** : πρέπει να είναι υποχρεωτικά ακαριαίοι και να βρίσκονται μεταξύ των ορίων [-32767,32767]

Εντολές:

- > Εκχώρηση
- > Απόφασης
- > Επανάληψης
- > Εισόδου
- > Εξόδου

Υποπρογράμματα:

- > Συναρτήσεις
- > Διαδικασίες

Μετάδοση Παραμέτρων:

- > Με σταθερή τιμή: **in**
- > Με αναφορά: **inout**

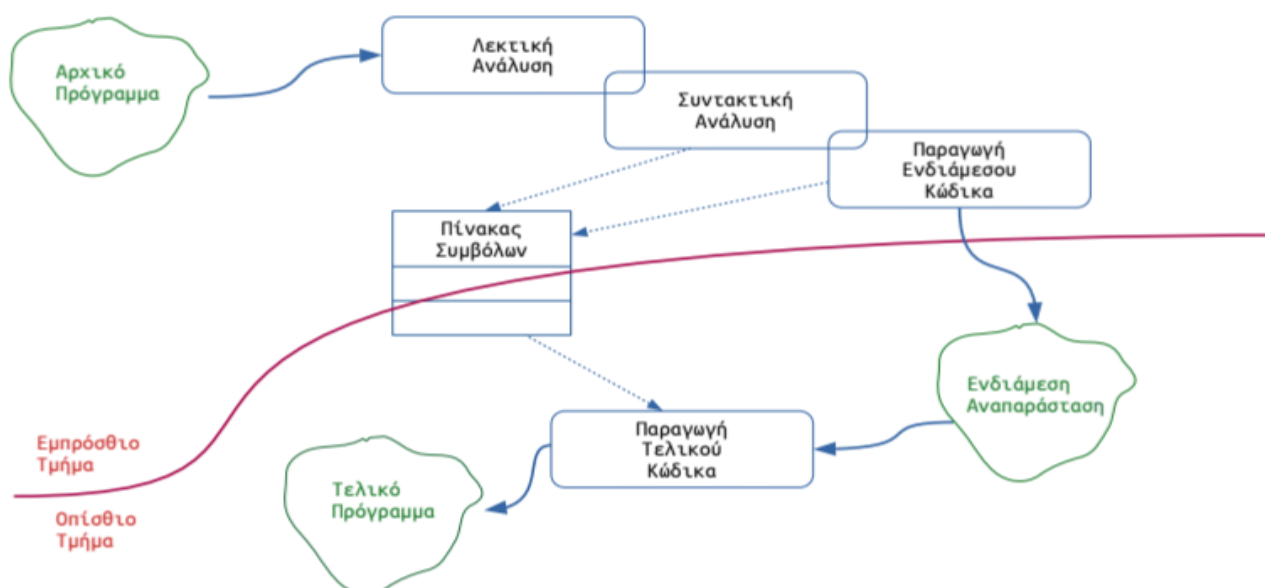
1.2 ΠΕΡΙΓΡΑΦΗ ASSEMBLY RISC-V

Ο **RISC-V** είναι ένας επεξεργαστής της τεχνολογίας RISC , διαθέτει 32 καταχωρητές για ακεραίους αριθμούς και 32 καταχωρητές για αριθμούς κινητής υποδιαστολής. Τα παραγόμενα αρχεία τελικού κώδικα αποθηκεύονται με την μορφή « **filename.s** » και εκτελούνται με την εντολή : **java -jar rars_46ab74d.jar filename.s**

1.3 ΜΕΤΑΦΡΑΣΤΗΣ GREEK++

Στην παρούσα εργασία δημιουργήσαμε το αρχείο **compiler.py** το οποίο λειτουργεί ως μεταφραστής της γλώσσας Greek++ σε γλώσσα μηχανής Assembly. Για να πραγματοποιήσουμε την μετάφραση ακολουθήσαμε τα εξής στάδια:

- Λεκτική Ανάλυση
- Συντακτική Ανάλυση
- Παραγωγή Ενδιάμεσου Κώδικα
- Σημασιολογική Ανάλυση
- Πίνακας Συμβόλων
- Παραγωγή Τελικού Κώδικα



2. ΛΕΚΤΙΚΗ ΑΝΑΛΥΣΗ

Ο Λεκτικός αναλυτής αποτελεί την πρώτη φάση της μεταγλώττισης κατά την οποία το αρχικό πρόγραμμα διαιρείται σε μία σειρά από λεκτικές μονάδες. **Στόχος** είναι η κατασκευή ενός αυτομάτου, το οποίο θα αναγνωρίζει τις λεκτικές μονάδες του αρχικού προγράμματος.



2.1 ΛΕΚΤΙΚΗ ΜΟΝΑΔΑ

Μία λεκτική μονάδα είναι ένα αντικείμενο, το οποίο περιέχει 3 πεδία, το πρώτο είναι η συμβολοσειρά που αναγνωρίστηκε **recognised_string**, το δεύτερο εντάσσει την συμβολοσειρά σε ένα **family** ανάλογα με τον τύπο του και το τρίτο αποθηκεύει τον αριθμό γραμμής που αναγνωρίστηκε η λεκτική μονάδα. Η υλοποίηση της δομής αυτής πραγματοποιήθηκε με την κλάση **Token**.

```
class Token:
    def __init__(self, family, recognised_string, line_number):
        self.family = family
        self.recognised_string = recognised_string
        self.line_number = line_number
```

Η κλάση αυτή αποτελείται από τα **πεδία**:

- **family:**
 - **Identifier:** ονόματα μεταβλητών, συναρτήσεων, διαδικασιών, σταθερών, οποιαδήποτε ονομασία
 - **number:** αριθμητικές σταθερές
 - **keyword:** περιέχει τις λέξεις κλειδιά της γλώσσας
 - **addOperator:** προσθετικοί αριθμητικοί τελεστές: +, -
 - **mulOperator:** πολλαπλασιαστικοί αριθμητικοί τελεστές: *, /
 - **relOperator:** λογικοί τελεστές, π.χ.: =, >=, <, <>, >, <=
 - **assignment:** τελεστής εκχώρησης (:=)
 - **delimiter:** διαχωριστές, π.χ.: ,, ., ;
 - **groupSymbol:** σύμβολα ομαδοποίησης, π.χ.: (,), {, }, [,]
- **recognised_string**
- **line_number**

Για την πρόσβαση και την διαχείριση τους χρησιμοποιήθηκαν οι εξής βοηθητικές **συναρτήσεις**:

```
def get_recognised_string(self):
    ...
def get_family(self):
    ...
def get_line_number(self):
    ...
def set_family(self, family):
```

```

...
def set_recognised_string(self, recognised_string):
...
def set_line_number(self, line_number):
...
def __str__(self):
...

```

2.2 ΒΑΣΙΚΕΣ ΒΟΗΘΗΤΙΚΕΣ ΣΥΝΑΡΤΗΣΕΙΣ ΓΙΑ ΤΟΝ LEXER

```
def set_index(new_index):
```

Θέτει νέα τιμή στον **index**.

```
def get_index():
```

Επιστρέφει την global μεταβλητή **index** (τρέχουσα θέση στον χαρακτήρα της γραμμής).

```
def get_test_line():
```

Επιστρέφει την global μεταβλητή **test_line** (τρέχουσα γραμμή).

```
def count_lines(file_path):
```

Διαβάζει αρχείο και επιστρέφει τον αριθμό γραμμών του. Εμφανίζει σφάλμα αν είναι κενό ή δεν υπάρχει.

```
def return_line(file_path,current_line):
```

Επιστρέφει το περιεχόμενο της γραμμής **current_line** από το αρχείο.

```
def next_char(current_line, index):
```

Επιστρέφει τον επόμενο χαρακτήρα στην τρέχουσα γραμμή.

```
def has_next_char(current_line,index):
```

Επιστρέφει **True** αν υπάρχει επόμενος χαρακτήρας.

2.3 ΚΛΑΣΗ LEX

```

class Lex:
    def __init__(self, current_line, file_path,token : Token):
        self.current_line = current_line
        self.file_path = file_path
        self.token = token

```

Η κλάση **Lex** αποτελεί τον πυρήνα ενός λεκτικού αναλυτή (lexer), ενός σημαντικού(1^{ου}) σταδίου στη διαδικασία μεταγλώττισης ενός προγράμματος. Ο ρόλος της είναι να διαβάζει τον πηγαίο κώδικα γραμμή προς γραμμή, να εντοπίζει χαρακτήρες και σύμβολα, και να τα μετατρέπει σε λεκτικά σύμβολα (tokens) που περιγράφουν τη συντακτική λειτουργία κάθε στοιχείου, όπως αριθμούς, λέξεις-κλειδιά, αναγνωριστικά, τελεστές και άλλα. Πιο συγκεκριμένα, οι κύριες μέθοδοι της είναι :

```
def lexer(self,processing_line_number,file_path):
```

Διαβάζει μία συγκεκριμένη γραμμή από αρχείο και ξεκινά τη λεκτική ανάλυση από αυτήν

```
def set_token(self,family,recognised_string):
```

Ορίζει τα χαρακτηριστικά του token (family, recognised_string, test_line)

```
def error(self, message):
```

Εμφανίζει μήνυμα σφάλματος.

```
def rem(self, start, file_path, line_number, index):
```

Εντοπίζει σφάλματα σε μη κλειστά ή ανοιγμένα μπλοκ σχολίων

```
def skip(self):
```

Αγνοεί τα σχόλια

```
def dig(self, start, processing_line):
```

Δημιουργεί **token** τύπου **integer** ή εμφανίζει σφάλμα αν είναι μη έγκυρος

```
def idk(self, start, processing_line):
```

Δημιουργεί **token** τύπου **keyword** ή **id**

```
def addOperator(self, start):
```

Δημιουργεί **token** για πρόσθεση ή αφαίρεση (+, -)

```
def mulOperator(self, start):
```

Δημιουργεί **token** για πολλαπλασιασμό ή διαίρεση (*, /)

```
def groupSymbol(self, start):
```

Δημιουργεί **token** για συμβολισμούς ομαδοποίησης όπως (,), [,]

```
def delimiter(self, start):
```

Δημιουργεί **token** για οριοθέτες όπως , ή ;

```
def asgn_Ref(self, start):
```

Δημιουργεί **token** για ανάθεση με αναφορά (%)

```
def asgn(self, start, processing_line):
```

Ελέγχει αν υπάρχει ανάθεση (:=) και δημιουργεί **token**

```
def relOperator(self, start, processing_line):
```

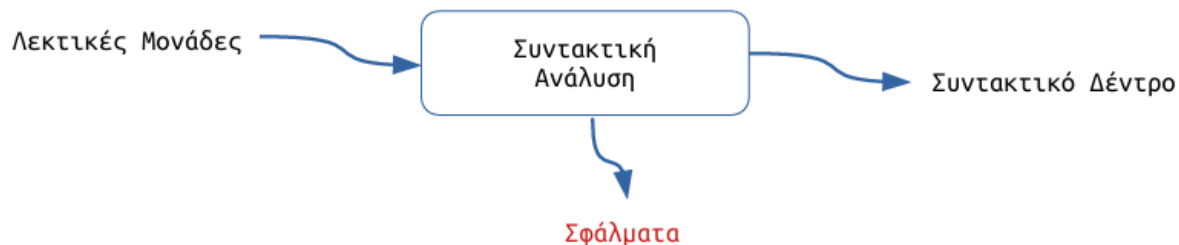
Αναγνωρίζει συγκριτικούς τελεστές όπως <, <=, >=, =, <>

```
def auto(self, start, processing_line):
```

Κεντρική συνάρτηση του **Lex** που επεξεργάζεται χαρακτήρες και βασιζόμενη στον χαρακτήρα που εξετάζει, αποφασίζει σε ποια κατηγορία ανήκει και καλεί τις αντίστοιχες βοηθητικές συναρτήσεις για την δημιουργία **token**.

3. ΣΥΝΤΑΚΤΙΚΗ ΑΝΑΛΥΣΗ

Στη 2^η φάση για τη δημιουργία ενός μεταφραστή δημιουργήσαμε τον **Συντακτικό Αναλυτή**. Ρόλος του είναι ο έλεγχος της ορθότητας ενός προγράμματος με βάση την γραμματική της γλώσσας και συγκεκριμένα εδώ της **Greek++** και παράγει κώδικα στις επόμενες φάσεις της ανάπτυξης. Συγκεκριμένα, ο συντακτικός αναλυτής διαβάζει μια προς μια τις λεκτικές μονάδες από τον **Lex** (Λεκτικό αναλυτή), τον οποίο είδαμε προηγουμένως και στη συνέχεια τις ελέγχει με τη σειρά εμφάνισής τους. Έτσι, θα υλοποιηθεί ως μία κλάση η οποία σχετίζεται με την κλάση **Lex** και την κλάση **Token**.



Αν εντοπίσει απόκλιση από τη γραμματική, παράγει **συντακτικό σφάλμα** και διακόπτει τη μεταγλώττιση.

3.1 ΚΛΑΣΗ PARSER

```
class Parser:
    global token
    global program_name

    def __init__(self, lexical_analyzer: Lex, generated_program: QuadList, symbol_table: SymbolTable):
        self.lexical_analyzer = lexical_analyzer
        self.generated_program = generated_program
        self.symbol_table = symbol_table
        self.token = None
```

ΔΟΜΗ ΤΗΣ ΚΛΑΣΗΣ PARSER

```
def program(self):
```

program: Ελέγχει αν το πρόγραμμα ξεκινά με τη λέξη «**πρόγραμμα**» και το όνομα του προγράμματος ανήκει στο **family id**.

```
def programblock(self):
```

programblock: Ελέγχει τη σωστή δομή του κυρίως προγράμματος από τα keyword "**αρχή_προγράμματος**" έως "**τέλος_προγράμματος**" και αναλύει τις δηλώσεις και εντολές που περιέχει.

```
def declarations(self):
```

declarations: Αναγνωρίζει δηλώσεις μεταβλητών που ξεκινούν με το keyword : "**δήλωση**" και εισάγει καθεμία στον πίνακα συμβόλων, ελέγχοντας για διπλές δηλώσεις.

```
def varlist(self):
```

varlist: Αναλύει και επιστρέφει λίστα **id** που χωρίζονται με ','.

```
def subprograms(self):
```

subprograms: Αναγνωρίζει υποπρογράμματα τύπου "**συνάρτηση**" ή "**διαδικασία**" και καλεί τις αντίστοιχες συναρτήσεις ανάλυσής τους.


```
def func(self):
```

func: Αναλύει δηλώσεις συναρτήσεων, ελέγχει σύνταξη, παραμέτρους και το σώμα τους.

```
def proc(self):
```

proc: Αναλύει τη δήλωση μιας διαδικασίας, αναγνωρίζει το όνομα και ελέγχει παραμέτρους και τον υπόλοιπο σκελετό.

```
def formalparlist(self):
```

formalparlist: Χρησιμοποιείται για την ορθή σύνταξη των παραμέτρων

```
def funcblock(self, func_name):
```

funcblock: Αναλύει το μπλοκ της συνάρτησης από τη λέξη "διαπροσωπεία" έως "τέλος_συνάρτησης".

```
def procblock(self, proc_name):
```

procblock: Αναλύει το μπλοκ της διαδικασίας από το keyword "διαπροσωπεία" έως το "τέλος_διαδικασίας".

```
def funcinput(self):
```

funcinput: . Καταχωρεί τις παραμέτρους εισόδου της συνάρτησης .

```
def funcoutput(self):
```

funcoutput: . Καταχωρεί τις παραμέτρους εξόδου της συνάρτησης.

```
def sequence(self):
```

sequence: Εκτελεί ανάλυση μιας ακολουθίας εντολών χωρισμένων με ‘;’.

```
def statement(self):
```

statement: Αναγνωρίζει και εκτελεί ανάλυση της κατάλληλης εντολής βάσει **keyword** ή **id** .

```
def assignment_stat(self):
```

assignment_stat: Αναλύει εντολή ανάθεσης τιμής με ‘:=’.

```
def if_stat(self):
```

if_stat: Αναλύει εντολή **εάν ... τότε ... εάν_τέλος** και διαχειρίζεται τις διακλαδώσεις.

```
def elsepart(self):
```

elsepart: Αναλύει το προαιρετικό σκέλος αλλιώς σε εντολή ‘εάν’.

```
def while_stat(self):
```

while_stat: Αναλύει δομή επανάληψης **όσο ... επανάλαβε ... όσο_τέλος**.

```
def do_stat(self):
```

do_stat: Αναλύει εντολή επανάληψης **επανάλαβε ... μέχρι**.

```
def for_stat(self):
```

for_stat: Αναλύει την εντολή επανάληψης **για ... έως ... επανάλαβε ... για_τέλος**.

```
def step(self):
```

step: Αναλύει τη φράση **με_βήμα** και επιστρέφει την έκφραση βήματος.

```
def print_stat(self):
```

print_stat: Αναλύει εντολή εξόδου **γράψε** .

```
def input_stat(self):
```

input_stat: Αναλύει εντολή εισόδου **διάβασε** με **id**.

```
def call_stat(self):
```

call_stat: Αναλύει κλήση διαδικασίας με το **εκτέλεσε** και στη συνέχεια **id**.

```
def id_tail(self):
```

idtail: Ελέγχει αν ένα αναγνωριστικό ακολουθείται από παράμετρο κλήσης.

```
def actualpars(self):
```

actualpars: Αναλύει τις παραμέτρους κλήσης συνάρτησης ή διαδικασίας.

```
def actualparlist(self):
```

actualparlist: Διαβάζει, ελέγχει και επιστρέφει όλες τις πραγματικές παραμέτρους μιας κλήσης.

```
def actualparitem(self):
```

actualparitem: Αναλύει παράμετρο με ή χωρίς αναφορά '%' σε κλήση συνάρτησης/διαδικασίας.

```
def condition(self):
```

condition: Αναλύει σύνθετες λογικές συνθήκες με τελεστές **ή**.

```
def boolterm(self):
```

boolterm: Αναλύει λογικούς όρους με τελεστές **και**.

```
def boolfactor(self):
```

boolfactor: Αναλύει στοιχειώδεις λογικές εκφράσεις ή σύνθετες και με **όχι**.

```
def expression(self):
```

expression: Αναλύει αριθμητικές εκφράσεις με προαιρετικό πρόσημο και τελεστές '+', '-', '*', '/', '^'.

```
def term(self):
```

term: Αναλύει όρους που συνδέονται με '*', '/', '^'.

```
def factor(self):
```

factor: Αναγνωρίζει integer, id ή εκφράσεις μέσα σε παρένθεση.

```
def relational_oper(self):
```

relational_oper: Επιστρέφει σχεσιακό τελεστή: '=', '<', '>', '>=', '<=', '<>'.

```
def add_oper(self):
```

add_oper: Ελέγχει και καταναλώνει πρόσθετους τελεστές '+', '-', '^'.

```
def mul_oper(self):
```

mul_oper: Αναγνωρίζει τελεστές '*', '/', '^' σε εκφράσεις.

```
def optional_sign(self):
```

optional_sign: Ελέγχει για προαιρετικό πρόσημο '+', '-', '^' πριν από όρο.

3.2 ΒΟΗΘΗΤΙΚΕΣ ΣΥΝΑΡΤΗΣΕΙΣ ΓΙΑ ΤΟΝ PARSER

```
def newTemp(self):
```

newTemp(self): Δημιουργεί και επιστρέφει νέο προσωρινό αναγνωριστικό μεταβλητής τύπου **T_i**.

```
def write_int(self, file=None):
```

write_int(self, file=None): Γράφει τις τετράδες (intermediate code) σε αρχείο ώστε αργότερα να γίνει της μορφής «**filename.int**».

```
def syntax_analyzer(self):
```

syntax_analyzer(self): Ξεκινά τη συντακτική ανάλυση, διαβάζει **token** και καλεί τη **program()**.

```
def error(self, message):
```

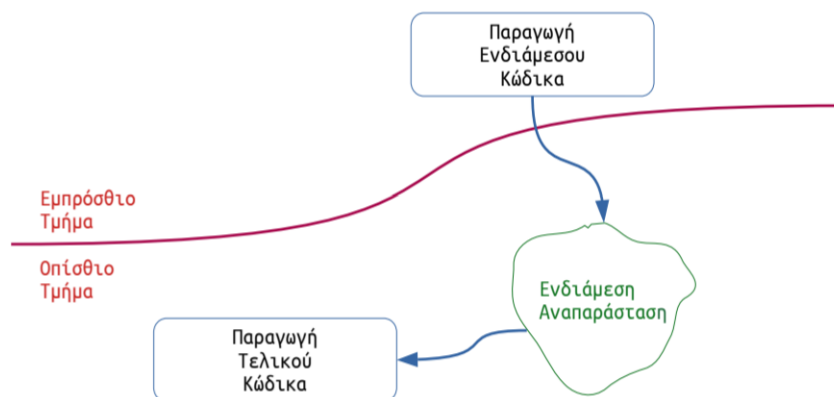
error(self, message): Εμφανίζει μήνυμα σφάλματος και τερματίζει τη μεταγλώττιση.

```
def get_token(self):
```

get_token(self): Παίρνει το επόμενο έγκυρο **token** από τον λεκτικό αναλυτή, αγνοώντας κενά και γραμμές.

4. ΠΑΡΑΓΩΓΗ ΕΝΔΙΑΜΕΣΟΥ ΚΩΔΙΚΑ

Η ενδιάμεση αναπαράσταση πρόκειται για μία γλώσσα υψηλού επιπέδου που βασίζεται σε τετράδες **Quads**, οι δομές της δεν είναι τόσο σύνθετες, όσο αυτές της αρχικής γλώσσας. Ο ενδιάμεσος κώδικας **Intermediate Code** παράγεται κατά την διάρκεια της συντακτικής ανάλυσης και αποθηκεύεται στο αρχείο « **filename.int** ».



Κάθε τετράδα ενδιάμεσου κώδικα αποτελείται από το label και τέσσερις operands. Για την αναπαράσταση κάθε τετράδας χρησιμοποιήθηκε η κλάση **Quad** η δομή της οποίας δίνεται παρακάτω:

```
class Quad:
    def __init__(self, label, op, op1, op2, op3):
        self.label = label
        self.op = op
        self.op1 = op1
        self.op2 = op2
        self.op3 = op3

    def __str__(self):
        return f"{self.label}: {self.op}, {self.op1}, {self.op2}, {self.op3}"
```

Για την παραγωγή των τετράδων χρησιμοποιήσαμε κάποιες βοηθητικές συναρτήσεις τις οποίες υλοποιήσαμε μέσα στην κλάση **QuadList**. Η QuadList έχει ως πεδία το **programList** το οποίο είναι μια global λίστα που αποθηκεύει τα quads που παράχθηκαν και το **quad_counter** που αποθηκεύει τον αριθμό των παραγόμενων Quads.

```
class QuadList:
    global programList
    global quad_counter

    def __init__(self, programList, quad_counter):
        self.programList = programList
        self.quad_counter = quad_counter
```

4.1 QUADLIST ΒΟΗΘΗΤΙΚΕΣ ΣΥΝΑΡΤΗΣΕΙΣ

```
def backpatch(self, list, z):
```

Διαβάζει μία μία τις τετράδες που σημειώνονται στη **programList** και για την τετράδα που αντιστοιχεί στην ετικέτα αυτή, συμπληρώνουμε το τελευταίο πεδίο της με το **label**.

```
def genQuad(self, operator, operand1, operand2, operand3):
```

Δημιουργεί μία νέα τετράδα, το πρώτο πεδίο της οποίας είναι το **operator** και τα τρία επόμενα τα **operand1**, **operand2**, **operand3**. Ο αριθμός της τετράδας προκύπτει από τον **quad_counter** αυξάνοντας την τιμή του κατά ένα.

```
def nextQuad(self):
```

Επιστρέφει την ετικέτα την επόμενης τετράδας που θα δημιουργηθεί, όταν κληθεί η **genQuad**.

4.2 GLOBAL ΒΟΗΘΗΤΙΚΕΣ ΣΥΝΑΡΤΗΣΕΙΣ

```
def mergelist(list1, list2):
```

Συνενώνει τις λίστες **list1** και **list2** σε μία λίστα.

```
def emptylist():
```

Δημιουργεί μία νέα κενή λίστα.

```
def makelist(i):
```

Δημιουργεί και επιστρέφει μία νέα λίστα η οποία έχει ως μοναδικό στοιχείο **i**.

4.3 ΠΑΡΑΓΩΓΗ ΕΝΔΙΑΜΕΣΟΥ ΚΩΔΙΚΑ

Για την παραγωγή του ενδιάμεσου κώδικα χρησιμοποιήσαμε τις παραπάνω συναρτήσεις στον κώδικα του συντακτικού αναλυτή. Συγκεκριμένα:

```
def programblock(self):
    ...
    if token.recognised_string == "αρχή_προγράμματος":
        ...
        # Intermediate_Code -> Generate Quad for main program
        self.generated_program.genQuad("begin_block", program_name, "_", "_")
        ...
        if token.recognised_string == "τέλος_προγράμματος":
            self.token = self.get_token()
        ...
    # Intermediate_Code -> halt fo main program
    self.generated_program.genQuad("halt", "_", "_", "_")
    # Intermediate_Code -> Generate Quad for end of main program
    self.generated_program.genQuad("end_block", program_name, "_", "_")
```

Στην **programblock** κλήθηκε η συνάρτηση **GenQuad** για την παραγωγή **Quad** για την αρχή και το τέλος του main προγράμματος.

```

def func(self):
    ...
    if token.recognised_string == "συνάρτηση":
        ...
        if token.family == "id":
            ...
            if token.recognised_string == "(":
                ...
                if token.recognised_string == ")":
                    ...
                    self.funcblock(func_name)
                    ...
                    # Intermediate_Code -> Generate Quad for function block end
                    self.generated_program.genQuad("end_block", func_name, "_", "_")
                    ...

def funcblock(self, func_name):
    ...
    if token.recognised_string == "διαπροσωπεία":
        ...

    if token.recognised_string == "αρχή_συνάρτησης":
        # Intermediate_Code -> Generate Quad for function block start
        self.generated_program.genQuad("begin_block", func_name, "_", "_")
        ...
        if token.recognised_string == "τέλος_συνάρτησης":
            ...

```

Για την παραγωγή Quad για την αρχή block συνάρτησης καλέσαμε την **GenQuad** στην συνάρτηση **funcblock** αμέσως μετά την αναγνώριση του string "αρχή_συνάρτησης". Αντίστοιχα για το τέλος block συνάρτησης καλέσαμε την **genQuad** στην συνάρτηση **func** αφού τελειώσει η funcblock με την αναγνώριση του string "τέλος_συνάρτησης".

```

def expression(self):
    ...
    if(self.optional_sign() == True and optional_sign == "-"):
        ...
        # Intermediate_Code -> Generate Quad for optional sign
        self.generated_program.genQuad(optional_sign, op1, op2, temp)
        ...
    while token.recognised_string in ["+", "-"]:
        ...
        # Intermediate_Code -> Generate Quad for addition/subtraction
        self.generated_program.genQuad(operator, op1, op2, temp)

```

Για του τελεστές + και – παράγονται τετράδες ενδιάμεσου κώδικα είτε για προσεσημασμένους αριθμούς είτε για πράξεις μεταξύ αριθμών στην συνάρτηση **expression**.

```
def term(self):
    ...
    while token.recognised_string in ["*", "/"]:
        ...
        # Intermediate_Code -> Generate Quad for multiplication/division
        self.generated_program.genQuad(operator, op1, op2, temp)
    ...
```

Για την παραγωγή Quad για τις πράξεις του πολλαπλασιασμού και διαίρεσης καλέσαμε την **genQuad** μέσα στην συνάρτηση **term**.

```
def actualpars(self):
    ...
    if token.recognised_string == "(":
        ...

    if token.recognised_string == ")":
        ...
        if old_line == new_line and last_func_proc in declared_func:
            ...
            # Intermediate_code -> Generate Quads for function call
            self.generated_program.genQuad("par", latest_function_ret, "ret", last_func_proc)
            self.generated_program.genQuad("call", last_func_proc, "_", "_")
        else:
            # Intermediate_code -> Generate Quad for function call
            self.generated_program.genQuad("call", last_func_proc, "_", "_")
    ...
```

Ο ενδιαμέσος κώδικας για το τέλος μίας συνάρτησης ή διαδικασίας παράγεται μέσα στην **actualpars**.

```
def actualparitem(self):
    ...
    if token.recognised_string == "%":
        ...
        if token.family == "id":
            ...
            # Intermediate_Code -> Generate Quad for ref parameter
            self.generated_program.genQuad("par", param, "ref", last_func_proc)
        ...
    else:
        ...
        if token.family == "id":
            ...
            # Intermediate_Code -> Generate Quad for cv parameter
            self.generated_program.genQuad("par", param, "cv", last_func_proc)
        ...
```

Οι παράμετροι των συναρτήσεων και των διαδικασιών περνιούνται ως τετράδες ενδιαμέσου κώδικα στην **actualparitem** ανάλογα με τον τρόπο μετάδοσης τους (αναφορά ή με τιμή).

```
def assignment_stat(self):
    ...
    # Intermediate_Code -> Generate Quads for assignment
    if str(source) == func_name:
        self.generated_program.genQuad(":= ", temp_list_T[-1], "_", target)
    else:
```

```

        self.generated_program.genQuad(":", source, "_", target)

    if target == func_name:
        func_return_found = True
        self.generated_program.genQuad("retv", "_", "_", source)
    ...

```

Για την ανάθεση μεταβλητών και την επιστροφή συνάρτησης παράγουμε Quads μέσα στην **assignment_stat**.

```

def if_stat(self):
    ...
    if token.recognised_string == "εάν":
        ...
        true_B, false_B = self.condition()

    if token.recognised_string == "τότε":
        ...
        self.generated_program.backpatch(true_B, self.generated_program.nextQuad())
        ...
        if_part = makelist(self.generated_program.nextQuad())
        # Intermediate_Code -> Generate Quad for if statement jump
        self.generated_program.genQuad("jump", "_", "_", if_part[0])
        self.generated_program.backpatch(false_B, self.generated_program.nextQuad())
        self.elsepart()
        self.generated_program.backpatch(if_part, self.generated_program.nextQuad())
    if token.recognised_string == "εάν_τέλος":
        ...

```

Αν η συνθήκη του **if_stat** είναι αληθής, τότε χρησιμοποιούμε τη λίστα true του **condition** με τη συνάρτηση **backpatch** για να παράγουμε την επόμενη τετράδα. Στη συνέχεια, αφού εκτελεστούν οι εντολές του if, δημιουργούμε μια λίστα με τη βοήθεια των **makelist** και **genquad** ώστε να μπορούμε να μεταβούμε στο τέλος της εντολής if. Αντίθετα, αν η συνθήκη είναι ψευδής, τότε χρησιμοποιούμε τη λίστα false του **condition** με την **backpatch** για να μεταβούμε στην τετράδα που ακολουθεί μετά την if.

```

def while_stat(self):
    ...
    if token.recognised_string == "όσο":
        Bquad = self.generated_program.nextQuad()
        ...
        true_B, false_B = self.condition()
    if token.recognised_string == "επανάλαβε":
        self.generated_program.backpatch(true_B, self.generated_program.nextQuad())
        ...
        # Intermediate_Code -> Generate Quad for while statement jump
        self.generated_program.genQuad("jump", "_", "_", Bquad)
        self.generated_program.backpatch(false_B, self.generated_program.nextQuad())
    if token.recognised_string == "όσο_τέλος":
        ...

```

Στη **whileStat**, αν η συνθήκη είναι αληθής, χρησιμοποιούμε τη λίστα true του **condition** με τη συνάρτηση **backpatch** για να μεταβούμε στην επόμενη τετράδα, όπου ξεκινά το σώμα της while. Αφού εκτελεστούν οι εντολές μέσα στη while, δημιουργούμε μια τετράδα με τη συνάρτηση **genquad** που μας επιστρέφει στην αρχή της επανάληψης. Αν όμως η συνθήκη είναι ψευδής, τότε χρησιμοποιούμε τη λίστα false του **condition** με την **backpatch** για να μεταβούμε στην τετράδα που ακολουθεί μετά τη λήξη της while.


```

def boolfactor(self):
    ...

    else:
        ...
        true_R = makelist(self.generated_program.nextQuad())
        # Intermediate_Code -> Generate Quad for relational operation
        self.generated_program.genQuad(rel_operation, expression1, expression2, "_")
        false_R = makelist(self.generated_program.nextQuad())
        # Intermediate_Code -> Generate Quad for jump
        self.generated_program.genQuad("jump", "_", "_", "_")

    return true_R, false_R

```

Στην τελευταία περίπτωση της **boolfactor** , αρχικά δημιουργούμε μια λίστα για τις περιπτώσεις όπου η συνθήκη είναι αληθής, χρησιμοποιώντας τη **makelist**. Στη συνέχεια, παράγουμε την αντίστοιχη τετράδα με τη **genquad**. Έπειτα, δημιουργούμε και τη λίστα για τις περιπτώσεις όπου η συνθήκη είναι ψευδής, πάλι με τη **makelist**, και παράγουμε μία ακόμα τετράδα με τη **genquad** που περιλαμβάνει ένα άλμα jump, ώστε να καθορίζεται η σωστή ροή εκτέλεσης.

```

def for_stat(self):
    ...

    if token.recognised_string == "για":
        ...
        if token.family == "id":
            val1 = token.get_recognised_string()
            ...
            if token.recognised_string == "!=":
                ...
                start_val = self.expression()
                # Intermediate_Code -> Generate Quad for assignment
                self.generated_program.genQuad("=", start_val, "_", val1)
            if token.recognised_string == "έως":
                ...
                condQuad = self.generated_program.nextQuad()
                end_val = self.expression()
                ...
                true_list = makelist(self.generated_program.genQuad("<=", val1, end_val, "_"))
                false_list = makelist(self.generated_program.genQuad("jump", "_", "_", "_"))
                step = self.step()
                if token.recognised_string == "επανάλαβε":
                    ...
                    sQuad = self.generated_program.nextQuad()
                    ...
                    # Intermediate_Code -> Generate Quad for for statement
                    self.generated_program.genQuad("+", val1, step, val1)
                    self.generated_program.genQuad("jump", "_", "_", condQuad)

                if token.recognised_string == "για_τέλος":
                    ...
                    self.generated_program.backpatch(true_list, sQuad)
                    self.generated_program.backpatch(false_list, self.generated_program.nextQuad())

```

Ο έλεγχος ροής στη **for_stat** ξεκινά με την ανάθεση του val1. Έπειτα, δημιουργείται μια τετράδα για τη start_val . Αμέσως μετά παράγεται μια τετράδα σύγκρισης μεταξύ του μετρητή val1 και της τελικής τιμής end_val και αποθηκεύεται μέσω **makelist** στη λίστα true_list. Στο false_list αποθηκεύεται ένα ανοιχτό άλμα του οποίου ο προορισμός θα καθοριστεί αργότερα μέσω **backpatch**. Αν η συνθήκη είναι αληθής, με **backpatch(true_list, sQuad)** μεταφερόμαστε στην αρχή του σώματος της επανάληψης. Αν η συνθήκη αποδειχθεί ψευδής, η λίστα false_list, που περιέχει το άλμα εξόδου, ενημερώνεται με **backpatch(false_list, self.generated_program.nextQuad())** ώστε η εκτέλεση να συνεχιστεί μετά τη λήξη της επανάληψης.

```
def input_stat(self):
    ...
    if token.recognised_string == "διάβασε":
        ...
        # Intermediate_Code -> Generate Quad for input
        self.generated_program.genQuad("in", token.get_recognised_string(), "_", "_")
    ...
```

Περνάμε την input μεταβλητή ως μία τετράδα ενδιαμέσου κώδικα.

```
def print_stat(self):
    #print("\nPRINT_STAT\n")
    global token
    if token.recognised_string == "γράψε":
        self.token = self.get_token()
        print_out = self.expression()
        # Intermediate_Code -> Generate Quad for print
        self.generated_program.genQuad("out", print_out, "_", "_")
```

Περνάμε την output μεταβλητή ως μία τετράδα ενδιαμέσου κώδικα.

```
def proc(self):
    ...
    if token.recognised_string == "διαδικασία":
        ...

        if token.recognised_string == ")":
            ...
            # Intermediate_Code -> Generate Quad for procedure block end
            self.generated_program.genQuad("end_block", proc_name, "_", "_")
            ...

def procblock(self, proc_name):
    ...
    if token.recognised_string == "διαπροσωπεία":
        ...
        if token.recognised_string == "αρχή_διαδικασίας":
            # Intermediate_Code -> Generate Quad for procedure block start
            self.generated_program.genQuad("begin_block", proc_name, "_", "_")
        ...
```

Για την παραγωγή Quad για την αρχή block διαδικασίας καλέσαμε την **GenQuad** στην συνάρτηση **procblock** αμέσως μετά την αναγνώριση του string "αρχή_διαδικασίας". Αντίστοιχα για το τέλος block διαδικασίας καλέσαμε την **genQuad** στην συνάρτηση **proc** αφού τελειώσει η procblock με την αναγνώριση του string "τέλος_διαδικασίας".

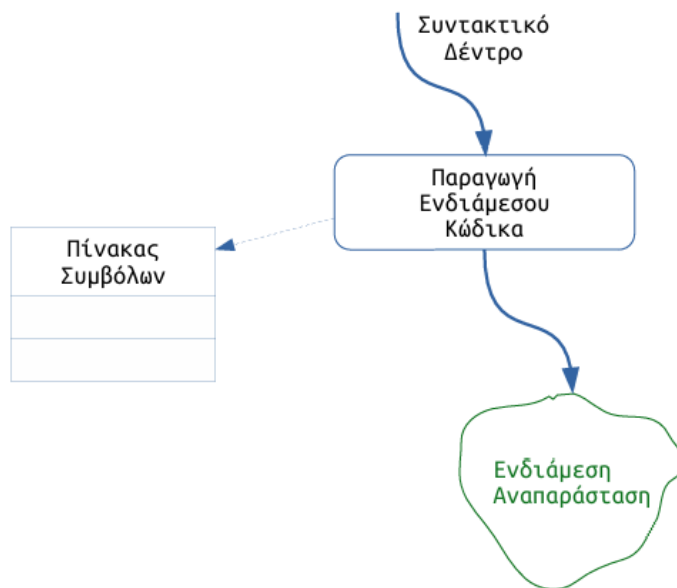
5. ΠΙΝΑΚΑΣ ΣΥΜΒΟΛΩΝ ΚΑΙ ΣΗΜΑΣΙΟΛΟΓΙΚΗ ΑΝΑΛΥΣΗ

Σε αυτή την ενότητα περιγράφεται η υλοποίηση και λειτουργία του πίνακα συμβόλων, ενός βασικού μηχανισμού που συνοδεύει τη συντακτική ανάλυση και την παραγωγή του ενδιάμεσου κώδικα. Ο πίνακας συμβόλων χρησιμεύει για την αποθήκευση όλων των πληροφοριών που σχετίζονται με τα συμβολικά ονόματα που χρησιμοποιούνται στο πρόγραμμα όπως μεταβλητές, συναρτήσεις και παράμετροι κλπ.

5.1 ΠΕΡΙΓΡΑΦΗ ΤΗΣ ΟΡΓΑΝΩΣΗΣ ΤΟΥ ΠΙΝΑΚΑ ΣΥΜΒΟΛΩΝ

Ο πίνακας οργανώνεται σε επίπεδα **Scopes** και οντότητες **Entities**. Κάθε φορά που ξεκινά ένα νέο υποπρόγραμμα, όπως η `main`, μια συνάρτηση ή μια διαδικασία, καλείται η μέθοδος `enter_scope()` ώστε να δημιουργηθεί ένα νέο περιβάλλον ορατότητας. Όταν το αντίστοιχο μπλοκ τελειώσει, καλείται `exit_scope()` ώστε να απομακρυνθεί αυτό το πεδίο από τον πίνακα.

Μέσα σε κάθε **Scope** καταγράφονται οι δηλωμένες μεταβλητές (**Variable**), παράμετροι (**Parameter**), προσωρινές μεταβλητές (**TemporaryVariable**) που δημιουργούνται αυτόματα για τις εκφράσεις, αλλά και τα υποπρογράμματα (**Function, Procedure**). Για κάθε οντότητα αποθηκεύεται το όνομά της, ο τύπος, η θέση της στον πίνακα ενεργοποίησης (`buffer_offset`), καθώς και επιπλέον χαρακτηριστικά ανάλογα με τον τύπο (όπως ο αριθμός παραμέτρων ή το `starting_quad`). Σημαντικό χαρακτηριστικό της γλώσσας **Greek++** είναι η υποστήριξη **φωλιασμένων υποπρογραμμάτων**, κάτι που απαιτεί προσεκτική διαχείριση των **Scopes** και σωστή οργάνωση του πίνακα συμβόλων, ώστε να αποφεύγονται συγκρούσεις ονομάτων και να υποστηρίζεται η σωστή αναδρομική αναφορά.



Στο τέλος της επεξεργασίας κάθε συνάρτησης ή διαδικασίας, ενημερώνονται στο αντίστοιχο **Entity**:

- το **startingQuad** (μέσω `update_entity`),
- και το **frameLength**, δηλαδή το μέγεθος μνήμης που απαιτείται.

Μετά την ολοκλήρωση της επεξεργασίας του προγράμματος, το περιεχόμενο του πίνακα συμβόλων καταγράφεται σε αρχείο `filename.sym` και εκτυπώνονται και στην κονσόλα.

5.2 ΥΛΟΠΟΙΗΣΗ ΤΟΥ ΠΙΝΑΚΑ ΣΥΜΒΟΛΩΝ

Για την δημιουργία του πίνακα συμβόλων δημιουργήσαμε τις παρακάτω κλάσεις:

- **Entity:** Η βασική υπερκλάση από την οποία κληρονομούν όλες οι υπόλοιπες οντότητες του πίνακα συμβόλων. Αντιπροσωπεύει κάθε είδους αναγνωριστικό που μπορεί να δηλωθεί στο πρόγραμμα (μεταβλητές, παράμετροι, υποπρογράμματα κ.λπ.).
- **Variable:** Κληρονομεί από την **Entity** και χρησιμοποιείται για την αναπαράσταση απλών μεταβλητών. Κάθε μεταβλητή διαθέτει όνομα, τύπο (**data_type**) και μετατόπιση (**offset**) εντός του πλαισίου ενεργοποίησης (**activation_record**). **TemporaryVariable:** Παράγει αυτόματα μεταβλητές για ενδιάμεσες τιμές σε εκφράσεις (π.χ. $T_1 = a + b$), επεκτείνοντας την **Variable**.
- **FormalParameter:** Αντιπροσωπεύει έναν τυπικό παράμετρο μίας συνάρτησης ή διαδικασίας. Περιλαμβάνει το όνομα, τον τύπο και τη λειτουργία της παραμέτρου (**mode: in, inout, ref**).
- **Parameter:** Επεκτείνει την **FormalParameter** προσθέτοντας και το **offset** στο **activation_record**.
- **Procedure:** Οντότητα που αναπαριστά μια διαδικασία. Περιλαμβάνει το **όνομα**, την **αρχική τετράδα (startingQuad)**, το μήκος(**frameLength**) και λίστες με τις παραμέτρους της.
- **Function:** Επεκτείνει την **Procedure** και προσθέτει τον τύπο επιστροφής (**data_type**). Αντιπροσωπεύει μια συνάρτηση με δυνατότητα επιστροφής τιμής.
- **Library:** Μια βοηθητική κλάση που χρησιμοποιείται για τη διατήρηση των οντοτήτων του πίνακα, ταξινομημένες ανά επίπεδο ορατότητας (**scope level**), για μεταγενέστερη χρήση, όπως π.χ. στην παραγωγή τελικού κώδικα.
- **Scope:** Αναπαριστά ένα πεδίο ορατότητας (**scope**), π.χ. για μια συνάρτηση ή διαδικασία. Κάθε **Scope** περιλαμβάνει λίστα με τις οντότητες που δηλώθηκαν σε αυτό, καθώς και έναν μετρητή **buffer** για τη διαχείριση της μετατόπισης στον **activation_record**.
- **SymbolTable:** Η κεντρική κλάση που διαχειρίζεται όλους τους πίνακες συμβόλων. Υποστηρίζει λειτουργίες όπως:
 1. **enter_scope() / exit_scope():** για είσοδο και έξοδο από **Scopes**
 2. **add_entity():** για την καταχώρηση νέων οντοτήτων
 3. **get_entity():** για την αναζήτηση οντοτήτων
 4. **update_entity():** για την ενημέρωση π.χ. **startingQuad** ή **frameLength**
 5. **print_symbol_table():** για την αποτύπωση της δομής του πίνακα σε αρχείο .sym

Η **SymbolTable** είναι η κύρια στην οποία χρησιμοποιούμε τις υπόλοιπες.

5.3 ΕΝΗΜΕΡΩΣΗ ΤΗΣ ΚΛΑΣΗΣ PARSER ΓΙΑ ΤΟΝ ΠΙΝΑΚΑ ΣΥΜΒΟΛΩΝ

Επίσης, χρειάστηκε να προσθέσουμε κώδικα στην κλάση **Parser** αφού χρειάζεται να πάρουμε την τελική παραγωγή του ενδιάμεσου κώδικα και να τον επεξεργαστούμε.

Πιο συγκεκριμένα επηρεάστηκαν οι παρακάτω συναρτήσεις της κλάσης Parser:

- **programblock:**
 - Χρήση της **enter_scope()** για να ξεκινήσει ένα νέο **scope** για το κυρίως πρόγραμμα.
 - Καταχώρηση της κύριας διαδικασίας (**Procedure**) στον πίνακα με **add_entity()**.
 - Ενημέρωση της με **update_entity()** για τα πεδία **startingQuad** και **frameLength**.
 - Κλήση της **exit_scope()** στο τέλος για απομάκρυνση του **scope**.
- **declarations:** Για κάθε μεταβλητή που δηλώνεται, γίνεται:
 - Έλεγχος διπλής δήλωσης με **get_entity()**.
 - Καταχώρηση μεταβλητής με **add_entity(Variable(...))**.
 - Ενημέρωση του **buffer** για τον υπολογισμό του **offset**.

- **func:**
 - Εισαγωγή της συνάρτησης στον πίνακα με `add_entity(Function(...))`.
 - Είσοδος σε νέο scope με `enter_scope()`.
 - Καταγραφή των παραμέτρων με `add_entity(Parameter(...))`.
 - Ενημέρωση της συνάρτησης για τον αριθμό και τύπο παραμέτρων.
 - Ενημέρωση της συνάρτησης στο τέλος με `update_entity()` για `frameLength` και `startingQuad`.
 - Έξοδος από το scope με `exit_scope()`.
- **funcblock:**
 - Ενημέρωση της συνάρτησης στο symbol table με `update_entity()` για `startingQuad`.
- **funcinput και funcoutput:**
 - Ενημέρωση των παραμέτρων μέσα στον πίνακα συμβόλων για το mode (in ή inout), χρησιμοποιώντας `get_entity()`.
- **expression, term και factor:**
 - Όταν δημιουργείται προσωρινή μεταβλητή (`newTemp()`), προστίθεται στον πίνακα συμβόλων ως `TemporaryVariable`.
 - Ενημέρωση του buffer για να καθοριστεί σωστά το offset της μεταβλητής.
- **actualpars, actualparitem:**
 - Χρήση της `add_entity(Parameter(...))` για καταγραφή actual parameters με `cv` ή `ref` mode και έλεγχος για μη δηλωμένες παραμέτρους με `get_entity()`.
- **assignment_stat:**
 - Έλεγχος ύπαρξης μεταβλητής στον πίνακα συμβόλων με `get_entity()`.
 - Σε περίπτωση που η μεταβλητή είναι το όνομα της συνάρτησης, προστίθεται quad τύπου `retv`.
- **call_stat:**
 - Έλεγχος ύπαρξης της διαδικασίας στον πίνακα συμβόλων με `get_entity()`.
 - Ορισμός της τελευταίας διαδικασίας για χρήση στο `actualpars`.
- **proc:**
 - Δημιουργία οντότητας τύπου `Procedure` με `add_entity()`.
 - Δημιουργία νέου scope με `enter_scope()`.
 - Καταχώρηση των παραμέτρων ως `Parameter` και προσθήκη στον πίνακα συμβόλων.
 - Τερματισμός με `exit_scope()` και ενημέρωση entity με `update_entity()`.
- **procblock:**
 - Ενημέρωση procedure με `update_entity()` για `startingQuad`.

5.4 ΣΥΜΠΕΡΑΣΜΑ ΤΗΣ ΛΕΙΤΟΥΡΓΙΑΣ ΤΟΥ ΠΙΝΑΚΑ ΣΥΜΒΟΛΩΝ

Συμπεραίνουμε πως με όλες τις παραπάνω συναρτήσεις, το **SymbolTable** χρησιμοποιείται για:

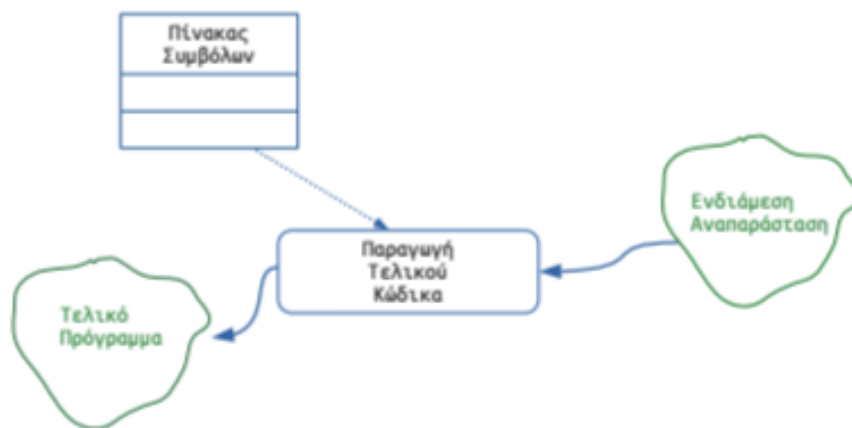
- **Καταχώρηση** νέων οντοτήτων (μεταβλητές, συναρτήσεις, παράμετροι, προσωρινές).
- **Οργάνωση** της ορατότητας μέσω scopes.
- **Έλεγχο** ύπαρξης και αποφυγή επαναδήλωσης.
- **Υπολογισμό offset** με χρήση του buffer.

5.5 ΣΗΜΑΣΙΟΛΟΓΙΚΗ ΑΝΑΛΥΣΗ

Στο πλαίσιο της σημασιολογικής ανάλυσης, αξιοποιώντας τον πίνακα συμβόλων, υλοποιήσαμε στην κλάση **Parser** μια σειρά από κρίσιμους ελέγχους για την ορθότητα του κώδικα. Πιο συγκεκριμένα, διασφαλίζεται ότι κάθε συνάρτηση περιλαμβάνει τουλάχιστον μία εντολή `return`, ότι δεν υπάρχει `return` μέσα σε διαδικασία, καθώς και ότι κατά την ανάθεση (**assignStat**) δεν γίνεται χρήση συναρτήσεων ως μεταβλητές. Αντίστοιχοι έλεγχοι πραγματοποιούνται και στις συναρτήσεις και διαδικασίες.

6. ΠΑΡΑΓΩΓΗ ΤΕΛΙΚΟΥ ΚΩΔΙΚΑ

Η παραγωγή του τελικού κώδικα είναι το στάδιο της μεταγλώττισης, όπου παράγεται ο κώδικας σε γλώσσα μηχανής. Ο τελικός κώδικας προκύπτει από τον ενδιάμεσο κώδικα με τη βοήθεια του πίνακα συμβόλων. Συγκεκριμένα, από κάθε εντολή ενδιάμεσου κώδικα προκύπτει μία σειρά εντολών τελικού κώδικα, η οποία για να παραχθεί ανακτά πληροφορίες από τον πίνακα συμβόλων.



Για την Greek++ θα παραγάγουμε τελικό κώδικα σε συμβολική γλώσσα μηχανής (assembly code) του επεξεργαστή RISC-V. Για την υλοποίηση αυτή δημιουργήσαμε μια νέα κλάση την **FinalCode**. Το παραγόμενο αρχείο τελικού κώδικα θα είναι της μορφής « **file_name.s** »

```
class FinalCode:
    def __init__(self, file_path, symbol_table: SymbolTable):
        self.file_path = file_path
        self.final_code = []
        self.symbol_table = symbol_table
```

6.1 ΒΟΗΘΗΤΙΚΕΣ ΣΥΝΑΡΤΗΣΕΙΣ ΤΕΛΙΚΟΥ ΚΩΔΙΚΑ

```
def gnlvcode(self, v):
```

gnlvcode: Παράγει τον τελικό κώδικα που επιτρέπει την προσπέλαση μιας πληροφορίας η οποία είναι αποθηκευμένη στο εγγράφημα δραστηριοποίησης κάποιου προγόνου της τρέχουσας συνάρτησης ή διαδικασίας, την ώρα που μεταφράζεται ο κώδικας.

```
def loadvr(self, v, reg):
```

loadvr: δημιουργεί τελικό κώδικα που διαβάζει την τιμή μιας μεταβλητής από τη μνήμη και τη μεταφέρει σε έναν καταχωρητή.

```
def storevr(self, reg, v):
```

storevr: εκτελεί την αντίστροφη λειτουργία από την loadvr, δηλαδή παράγει κώδικα που αποθηκεύει στη μνήμη την τιμή μιας μεταβλητής, η οποία αυτή τη στιγμή βρίσκεται σε καταχωρητή.

```
def produce(self, line):
```

produce: Δημιουργεί μια νέα γραμμή τελικού κώδικα.

```
def get_subprogram_label(self, name, subprogram_label_list):
```

get_subprogram_label: Επιστρέφει το label του τελικού κώδικα του subprogram name.

```
def get_entity(self, v):
```

get_entity: Επιστρέφει το entity v από το final_code_library

```
def gen_label(self):
```

gen_label: Επιστρέφει το επόμενο label τελικού κώδικα.

6.2 ΥΛΟΠΟΙΗΣΗ ΤΕΛΙΚΟΥ ΚΩΔΙΚΑ

Η υλοποίηση του τελικού κώδικα πραγματοποιήθηκε με την συνάρτηση **final_countdown** της κλάσης **FinalCode**. Συγκεκριμένα διατρέχουμε το global programList το οποίο περιέχει τον ενδιαμέσο κώδικα και για κάθε quad ανάλογα με τα operands του και με την βοήθεια των υπόλοιπων συναρτήσεων της κλάσης FinalCode παράγουμε τις αντίστοιχες εντολές τελικού κώδικα.

```
def final_countdown(self):  
    ...  
    self.produce('        .data')  
    self.produce('    str_n1: .asciz "\\n"')  
    self.produce('        .text')  
    self.produce(f'{self.gen_label()}')  
    self.produce(f'        j Lmain ')  
    ...
```

quad.op == 'retv'	self.loadvr(quad.op3, 't1') self.produce(f' lw t0, -8(sp)') self.produce(f' sw t1, (t0)')
quad.op == 'begin_block' and quad.op1 == program_name	self.produce('Lmain:') self.produce(f' addi sp, sp, {main.frameLength}') self.produce(' mv gp, sp')
quad.op == 'end_block' and quad.op1 in declared_func	self.produce(f' lw ra, (sp)') self.produce(' jr ra')
quad.op == 'end_block' and quad.op1 in declared_proc	self.produce(f' lw ra, (sp)') self.produce(' jr ra')
quad.op == 'begin_block' and quad.op1 in declared_func	self.produce(f' sw ra, (sp)')
quad.op == 'begin_block' and quad.op1 in declared_proc and quad.op1 != program_name	self.produce(f' sw ra, (sp)')

quad.op == ':' and quad.op3 not in declared_func and quad.op3 not in declared_proc:	self.loadvr(quad.op1, 't1') self.storevr('t1', quad.op3)
quad.op == 'par' and quad.op2 == 'cv'	self.produce(f' addi fp, sp,{entity.frameLength}') self.loadvr(quad.op1, 't0') self.produce(f' sw t0, {-12 + 4 * par_counter }(fp)') par_counter += 1
quad.op == 'par' and quad.op2 == 'ref'	self.produce(f' addi fp, sp,{entity.frameLength}') par_element = self.get_entity(quad.op1) par = par_element.entity
quad.op == 'par' and quad.op2 == 'ret'	self.produce(f' addi fp, sp,{entity.frameLength}') self.produce(f' addi t0, sp, {-entity.offset}') self.produce(f' sw t0, -8(fp)')
<i>quad.op == add_oper or mul_oper</i>	self.loadvr(quad.op1, 't1') self.loadvr(quad.op2, 't2') self.produce(f' {op} t1, t1, t2') self.storevr('t1', quad.op3)
quad.op == 'jump'	self.produce(f' j L{quad.op3}')
quad.op == rel_oper	self.loadvr(quad.op1, 't1') self.loadvr(quad.op2, 't2') self.produce(f' {cond_jump} t1, t2, L{quad.op3}')
quad.op == 'call' and called == caller	self.produce(' lw t0, -4(sp)') self.produce(' sw t0, -4(fp)') self.produce(f' addi sp, sp, {called_entity.frameLength}') self.produce(f' jal {label[:-1]}') self.produce(f' addi sp, sp, {-called_entity.frameLength}')
quad.op == 'call' and called != caller	self.produce(f' sw sp, -4(fp)') self.produce(f' addi sp, sp, {called_entity.frameLength}') self.produce(f' jal {label[:-1]}') self.produce(f' addi sp, sp, {-called_entity.frameLength}')
quad.op == 'out'	self.loadvr(quad.op1, 'a0') self.produce(f' li a7, 1') self.produce(f' ecall') self.produce(f' la a0, str_nl') self.produce(f' li a7, 4') self.produce(f' ecall')
quad.op == 'halt'	self.produce(' li a0, 0') self.produce(' li a7, 93') self.produce(' ecall')
quad.op == 'in'	self.produce(f' li a7, 5') self.produce(' ecall') self.storevr('a0', quad.op1)

6.3 ΠΑΡΑΔΕΙΓΜΑ ΕΚΤΕΛΕΣΗΣ ΠΑΡΑΓΟΜΕΝΟΥ ΑΡΧΕΙΟΥ ΣΤΟΝ RARS

Αρχικά τρέξαμε από το compiler μας με την εντολή **python compiler.py counter_11.gr**. Από την εκτέλεση αυτή παράχθηκαν τα αρχεία:

- **counter_11.int** : ενδιάμεσος κώδικας

- **counter_11.c**: ενδιάμεσος κώδικας σε c (συμβατό με προγράμματα χωρίς συναρτήσεις και διαδικασίες)
- **counter_11.sym** : πίνακας συμβόλων
- **counter_11.s** : τελικός κώδικας

The screenshot shows the RARS IDE with the assembly code for counter_11.s. The main window displays the assembly code with columns for instruction number, address, code, basic block, and source. The right panel shows the registers and their values. The bottom panel shows the messages and run I/O.

Registers	Floating Point	Control and Status	Value
zero			0x00000000
ra			0x00000000
sp			0x7ffffff0
fp			0x7ffffff0
tp			0x00000000
t0			0x00000000
t1			0x00000000
t2			0x00000000
t3			0x00000000
t4			0x00000000
t5			0x00000000
t6			0x00000000
t7			0x00000000
sl			0x00000000
sl2			0x00000000
sl3			0x00000000
sl4			0x00000000
sl5			0x00000000
sl6			0x00000000
sl7			0x00000000
sl8			0x00000000
sl9			0x00000000
sl10			0x00000000
sl11			0x00000000
sl12			0x00000000
sl13			0x00000000
sl14			0x00000000
sl15			0x00000000
sl16			0x00000000
sl17			0x00000000
sl18			0x00000000
sl19			0x00000000
sl20			0x00000000
sl21			0x00000000
sl22			0x00000000
sl23			0x00000000
sl24			0x00000000
sl25			0x00000000
sl26			0x00000000
sl27			0x00000000
sl28			0x00000000
sl29			0x00000000
sl30			0x00000000
sl31			0x00000000
pc			0x00400000

The screenshot shows the RARS IDE with the assembly code for counter_11.s. The main window displays the assembly code with columns for instruction number, address, code, basic block, and source. The right panel shows the registers and their values. The bottom panel shows the messages and run I/O.

Registers	Floating Point	Control and Status	Value
zero			0x00000000
ra			0x00000000
sp			0x7ffffff0
fp			0x7ffffff0
tp			0x00000000
t0			0x00000000
t1			0x00000000
t2			0x00000000
t3			0x00000000
t4			0x00000000
t5			0x00000000
t6			0x00000000
t7			0x00000000
sl			0x00000000
sl2			0x00000000
sl3			0x00000000
sl4			0x00000000
sl5			0x00000000
sl6			0x00000000
sl7			0x00000000
sl8			0x00000000
sl9			0x00000000
sl10			0x00000000
sl11			0x00000000
sl12			0x00000000
sl13			0x00000000
sl14			0x00000000
sl15			0x00000000
sl16			0x00000000
sl17			0x00000000
sl18			0x00000000
sl19			0x00000000
sl20			0x00000000
sl21			0x00000000
sl22			0x00000000
sl23			0x00000000
sl24			0x00000000
sl25			0x00000000
sl26			0x00000000
sl27			0x00000000
sl28			0x00000000
sl29			0x00000000
sl30			0x00000000
sl31			0x00000000
pc			0x00400000

Παράδειγμα εκτέλεσης counter_11.s στον RARS